Driver Drowsiness Detection using Adaptive Thresholding and OpenVINO

Shivam Tyagi
School of Computer Science and Engineering
Vellore Institute of Technology, Vellore, India
shivam.tyagi2026@vitstudent.ac.in

Akash A Gaddi
School of Computer Science and Engineering
Vellore Institute of Technology, Vellore, India
akash.gaddi2026@vitstudent.ac.in

Collaborator :Dr.J Saira Banu, School of Computer Science and Engineering, VIT Vellore
jsaribanu@vit.ac.in

***Abstract*—Detecting driver drowsiness from a single image is difficult because normal blinking, variations in eye geometry, illumination, eyeglasses, and head rotation can produce similar visual measurements. This paper presents a webcam-based prototype that combines an individually calibrated Eye Aspect Ratio (EAR), a compact OcuNet eye-state convolutional neural network, Mouth Aspect Ratio (MAR), head-pose estimation, and PERCLOS. A 60-second calibration period is used to estimate the driver's eye and mouth thresholds. Temporal counters then determine whether the observed behaviour persists long enough to trigger a warning. OpenVINO executes the supplied face detector and OcuNet intermediate-representation models, while MediaPipe provides facial landmarks. Controlled screenshots demonstrate calibration, normal monitoring, microsleep, yawning, and head-nod warning branches on an HP Victus laptop running Windows 11. Across five CPU screenshots, the interface displayed between 28.7 and 39.1 FPS, while face-detector calls took between 5.2 and 11.6 milliseconds. These readings are individual observations rather than average benchmark results. The repository does not contain OcuNet training arrays, training history, or labelled evaluation data. Therefore, this paper does not claim any independently measured accuracy or confusion matrix. The work documents the multi-cue implementation, available CPU/GPU observations, and a possible FPGA deployment architecture.**

***Index Terms*—Driver drowsiness detection, adaptive thresholding, EAR, MAR, PERCLOS, OpenVINO, edge-AI, FPGA.**

I. Introduction and Problem Statement

Reliable detection of driver drowsiness remains a challenging problem because fatigue develops over time and is not always visible in a single frame. Drowsy driving can reduce vigilance and delay a driver's response to unexpected situations. Earlier non-contact approaches examined eyelid closure, gaze direction, head movement, and facial expressions [1], [2]. Later studies incorporated facial dynamics [3], condition-dependent video representations [6], landmark features [7], and compact embedded models [20].

Previous reviews have also highlighted the trade-off between sensor intrusiveness, detection robustness, and computational cost [4], [5].

This project addresses a practical version of the problem: detecting visible fatigue-related indicators using a standard webcam, adapting the decision process to the individual driver, and executing the required computations on a laptop.

A fixed EAR threshold is inexpensive to calculate, but its behaviour can vary with eye shape, eyewear, illumination, and camera position. Similarly, a frame-level CNN can classify an eye crop as open or closed, but the classification alone does not distinguish a normal blink from a prolonged closure.

To address these issues, the proposed implementation combines geometric measurements with a learned eye-state representation. Temporal counters are also included so that one unusual frame does not immediately trigger a warning.

The system generates warnings based on sustained eye closure, prolonged mouth opening, downward head movement, and longer-window PERCLOS measurements.

This project is developed as a functional academic pro-to type. It has not been clinically validated or certified for automotive safety. The available evidence consists of implementation source code, supplied pretrained model files, and screenshots obtained during controlled operation. Since the repository does not contain a labelled trial dataset or end-to-end testing logs, the paper reports only the functionality supported by the available evidence.

II. Objectives and Contributions

A. Objectives

The main objectives of the project are:

- To identify visibly apparent drowsiness indicators in real time using a standard webcam.
- To estimate individual eye thresholds through a 60-second calibration period.
- To combine EAR, OcuNet eye-state classification, MAR, head orientation, and PERCLOS for generating warnings.
- To display the inference target, face-detection time, FPS, and cue values through a real-time interface.
- To examine CPU, GPU, and FPGA deployment feasibility before implementing additional hardware.

B. Main Contributions

The primary engineering contribution is the adaptive multi-cue decision mechanism. Individual eye and mouth measurements establish a personal baseline, while OcuNet supplies an additional learned eye-state estimate. Frame counters then require the visual evidence to persist before a warning is produced.

The second contribution is the transparency of the decision process. Each warning can be associated with displayed

measurements such as EAR, OcuNet probability, MAR, pitch angle, and PERCLOS.

The third contribution is the cross-platform execution demonstration of the same project source folder on Apple silicon. Separate runtime environments were needed to accommodate Windows-specific camera and audio assumptions.

III. Motivations for and Evaluating Feasibility of This Approach

The proposed approach is technically feasible on a laptop because the neural-network inputs are small and the remaining operations require relatively little computation.

The face detector accepts a 300×300-pixel input, while the eye CNN processes a 34×26 grayscale eye crop. The remaining calculations involve geometric ratios, frame counters, and six-point head-pose estimation.

The use of consumer-grade webcams and open-source libraries also makes the prototype economically accessible. However, deploying the system in an actual vehicle would require automotive-grade hardware and compliance with applicable road-safety, privacy, and functional-safety requirements.

From a social perspective, the system could provide an early warning when visible signs of fatigue appear. At the same time, excessive false alarms could reduce user confidence and lead drivers to ignore warnings.

Local processing is another advantage because continuous transmission of camera footage to cloud-service providers is not required.

Personal calibration is motivated by differences in eye opening, facial geometry, spectacle usage, and camera position. The system therefore estimates a baseline for the current user instead of assuming that a single EAR threshold is suitable for everyone.

The present work demonstrates an academic prototype only. Production certification and regulatory compliance have not been established.

IV. Literature Review and Analysis of Gaps in Current Technology

Driver drowsiness detection methods have gradually moved from manually selected visual measurements towards learned spatial and temporal representations.

Classical approaches remain useful because measures such as PERCLOS, EAR, blink duration, and head movement are relatively inexpensive and interpretable [1]–[8].

CNN-based and recurrent learning methods use richer representations of eyes and facial regions [9]–[17]. Graph-based approaches additionally model relationships between facial landmarks and different facial regions [18], [21].

Recent research also considers embedded implementation, lighting variation, class imbalance, and personalisation [20]–[26].

The literature survey is presented in Table I. The reported results are reproduced from the cited studies. They are not directly comparable because the datasets, labels, and evaluation protocols differ.

A. Research Gaps

Four research gaps influenced the design of the final implementation.

First, a fixed threshold does not account for differences in individual eye geometry and camera position.

Second, a single frame cannot reliably distinguish an ordinary blink from a prolonged eye closure.

Third, learned eye-state classifiers may become uncertain when glare, partial occlusion, or unusual head poses are present. Combining geometric and behavioural measurements can provide additional evidence.

Finally, high accuracy reported on research datasets does not automatically establish that a model satisfies the latency, power consumption, and transparency requirements of an edge-based application.

The proposed system addresses these issues through individual calibration, temporal persistence, multi-cue fusion, and decisions based on measurable outputs. Subject-independent validation remains future work.

B. Recent Trends

Recent literature explores relational graph models [21], transfer learning between eye and drowsiness datasets [22], generative methods for class imbalance [23], driver-specific modelling [24], and detection under complex lighting conditions [25].

Additional work has examined sensing technologies and physiological parameters for real-time drowsiness detection [26].

Although these directions provide useful opportunities for further investigation, they can require larger datasets and more computational resources than were available for the present prototype.

Note: The result column reproduces headline values reported by the original studies. These values are not directly comparable because the datasets, labels, and evaluation protocols differ.

V. Proposed System and Model Selection

The proposed system starts with a camera frame. OpenVINO performs face detection, and MediaPipe extracts facial landmarks. The resulting coordinates are used by three processing paths: eye analysis, mouth analysis, and head-pose estimation.

The outputs from these paths are combined during personal calibration and temporal fusion. A priority decision mechanism determines whether the system should display a normal monitoring interface or activate a warning.

A. Model Selection

Different approaches were considered before selecting the final hybrid model. Fixed EAR is inexpensive but sensitive to user and camera variation. Adaptive EAR with counters provides a personal threshold and closure-duration check. OcuNet supplies an additional learned eye-state probability.

LSTM, 3D CNN, and ST-GCN approaches were reviewed but not implemented because they require more data and computation.

TABLE I
SURVEY OF TWENTY JOURNAL PAPERS

Ref.	Paper / Journal	Year	Detection Problem and Method	Reported Result / Dataset	Advantages and Limitations
[1]	Real-Time Nonintrusive Monitoring and Prediction of Driver Fatigue	IEEE TVT	Non-contact fatigue monitoring using IR video, eyelid, gaze, head movement, and expression fusion.	Real-time multimodal validation using human fatigue recordings.	Multi-cue and non-contact approach; requires specialised IR hardware and older handcrafted features.
[2]	Real-Time System for Monitoring Driver Vigilance	IEEE T-ITS	PERCLOS, closure duration, blink, nodding, gaze, and fuzzy logic.	About 25 FPS; real-time driving video under favourable conditions.	Interpretable temporal cues; eyewear and daylight can reduce reliability.
[3]	Driver Drowsiness Detection Using Facial Dynamic Fusion Information and a DBN	IET ITS	Eye and mouth texture, facial landmarks, and a Dynamic Bayesian Network.	96.7% average result reported using custom multi-condition data.	Dynamic multi-cue fusion; sensitivity to pose and occlusion.
[4]	Driver Fatigue Detection Systems: A Review	IEEE T-ITS	Review of physiological, vehicle-based, and visual detection methods.	Comparative taxonomy.	Provides clear system classification; lacks a unified benchmark.
[5]	A Survey on State-of-the-Art Drowsiness Detection Techniques	IEEE Access	Survey of behavioural, vehicle, and physiological methods.	State-of-the-art synthesis.	Wide method coverage; evaluation protocols remain heterogeneous.

TABLE I (Continued)

Ref.	Paper / Journal	Year	Detection Problem and Method	Reported Result / Dataset	Advantages and Limitations
[6]	Condition-Adaptive Representation Learning for Driver Drowsiness	IEEE T-ITS	3D CNN with condition-aware adaptive fusion for lighting and eyewear variation.	Improved visual baselines on NTHU-DDD.	Condition-aware temporal features; relatively high 3D CNN computation.
[7]	Real-Time Driver-Drowsiness Detection Using Facial Features	IEEE Access	Facial landmarks with blink, closure duration, and yawning.	About 92% result reported using YawDD and driver videos.	Interpretable multi-cue design; performance may decline in dim light and with glasses.
[8]	Real-Time Classification for Autonomous Drowsiness Detection Using EAR	ESWA	EAR sequences with MLP, Random Forest, and SVM.	SVM about 94.9%; inter-subject result about 94.4%.	Low computational cost and personalisation; sensitive to pose and occlusion.
[9]	Real-Time Driver Drowsiness Detection Based on CNN	SN Computer Science	Custom CNN and VGG transfer learning for eye-state classification.	Best eye-state accuracy about 98.15%.	Strong eye-state classification; eye-state classification alone is not full drowsiness detection.
[10]	Real-Time Monitoring on Mobile Platforms Using 3D Neural Networks	NCA	Lightweight 3D convolutions for temporal inference on mobile platforms.	Improved mobile baseline with a sequence-window trade-off.	Mobile deployment focus; sequence inference has additional computational cost.

TABLE I (Continued)

Ref.	Paper / Journal	Year	Detection Problem and Method	Reported Result / Dataset	Advantages and Limitations
[11]	Real-Time Driver Fatigue Detection Based on a Recurrent Neural Network	Journal of Imaging	3D convolution and temporal recurrent network for face-sequence detection.	92.19% comparative accuracy reported on NTHU-DDD.	Temporal modelling; sensitive to sequence-window length.
[12]	Driver Fatigue Detection Using EM-CNN	CIN	MTCNN, EM-CNN, PERCLOS, and POM for eye and mouth fatigue detection.	93.623% accuracy reported using Biteda images.	Combines learned and temporal cues; lower specificity reported.
[13]	A Portable Fuzzy Driver Drowsiness Estimation System	Sensors	PERCLOS, closure duration, mouth opening, and fuzzy logic.	About 95.5% primary and 93.5% broader-test results reported.	Interpretable and Raspberry Pi capable; controlled sample is limited.
[14]	Efficient and Compact Face Descriptor for Drowsiness Detection	ESWA	Multi-level descriptor and feature selection.	Competitive benchmark performance on NTHU-DDD.	Compact representation; limited temporal context.
[15]	Deep CNN Models-Based Ensemble Approach to Driver Drowsiness Detection	NCA	Ensemble of CNN models and multiple cues.	About 85% final accuracy reported.	Rich feature fusion; higher computational requirement.

B. OcuNet Architecture

The supplied training script defines three sequential blocks of 3×3 convolutions using 16, 32, and 64 filters, respectively. Each block uses batch normalisation, ReLU activation, and 2×2 max pooling.

An adaptive average-pooling operation produces a feature vector of length 64. A fully connected layer maps the 64 features to 32 units, followed by ReLU activation, dropout with probability 0.25, and two output logits representing the "closed" and "open" classes.

The model contains approximately 25.7 thousand trainable parameters. The OpenVINO IR files occupy approximately 76 KB in the project directory.

The original training script specifies cross-entropy loss, AdamW optimisation, a cosine-based learning-rate decay schedule, and 25 training epochs. The best PyTorch checkpoint is converted into OpenVINO IR format.

However, the expected training arrays and training logs are not included in the repository. The supplied weights can be used by the application, but their original training result cannot be independently verified.

TABLE I (Continued)

Ref.	Paper / Journal	Year	Detection Problem and Method	Reported Result / Dataset	Advantages and Limitations
[16]	Using LSTM and CNNs for Driver Drowsiness Detection	AAP	Eye sequences with LSTM and ConvLSTM.	LSTM about 82%; ConvLSTM about 95–97%. Dataset contained 38 subjects with EEG-derived labels.	Temporal modelling with ground truth; simulated-driving conditions limit generalisation.
[17]	An Efficient Deep-Learning Approach for Drowsiness Detection	Applied Sciences	Adaptive EAR and yawn thresholds with transfer learning.	83%, 96%, and 97% across reported scenarios.	Direct personalisation study; transfer across protocols requires further examination.
[18]	Two-Stream Spatial-Temporal Graph CNNs for Drowsiness	IEEE TCYB	Facial-landmark graph with two-stream ST-GCN.	About 93.4% on YawDD and 92.7% on NTHU-DDD.	Strong temporal landmark modelling; more complex than EAR-based methods.
[19]	Eye Aspect Ratio for Real-Time Drowsiness Detection	Electronics	Landmarks and EAR threshold sweep.	Best threshold 0.18; about 96–97% on tested videos.	Demonstrates threshold sensitivity; mainly examines blinking and closure.
[20]	Embedded Drowsiness Detection Using CNN and MAR	Sensors	MediaPipe, CNN eye classifier, MAR, and NIR.	99.88% test result, 96.55% hardware result, and about 9–14 FPS.	Strong embedded hybrid design; hardware and operating conditions are platform-specific.

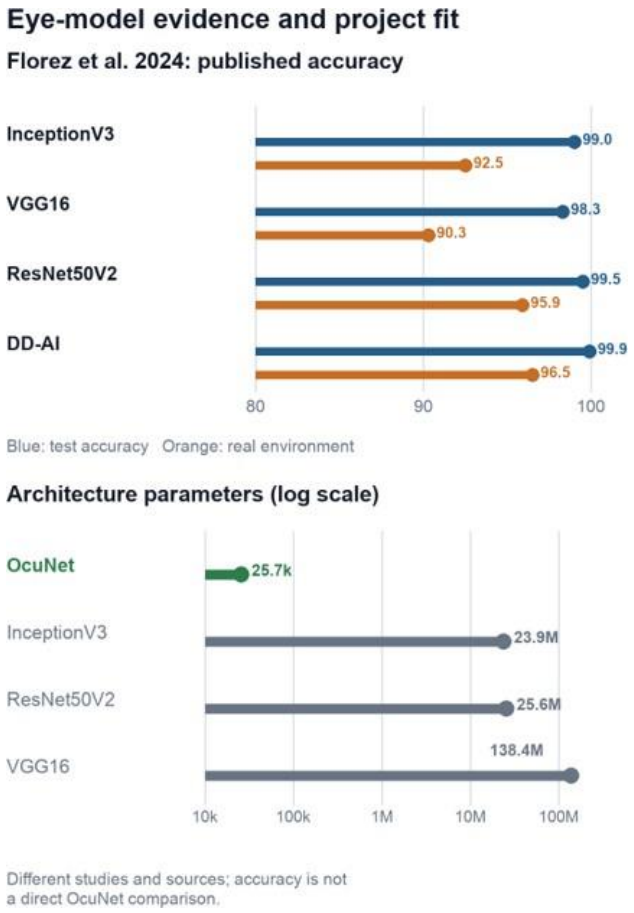


Fig. 1. Implemented multi-cue processing architecture.

C. Mathematical Decision Model

EAR is calculated using the vertical and horizontal distances between eye landmarks. MAR measures the vertical opening of the mouth relative to its horizontal width.

The mathematical measurements used by the implementation are shown in Fig. 2.

TABLE II
MODEL ALTERNATIVES AND FINAL SELECTION

Approach	Status	Observation and Decision
Fixed EAR	Literature baseline	Low cost, but sensitive to variation. Not selected alone.
Adaptive EAR	Implemented	Personal threshold and closure duration. Primary eye cue.
OcuNet CNN	Supplied model loaded	Learned eye-state probability from a 34×26 crop. Supporting cue.
LSTM, 3D CNN, ST-GCN	Reviewed	Higher data and compute requirements. Not implemented.
Hybrid model	Implemented	Adaptive EAR, OcuNet, MAR, pose, and PERCLOS. Final model.

$$EAR = \frac{|p_2 - p_6| + |p_3 - p_5|}{2|p_1 - p_4|} \tag{1}$$

$$MAR = \frac{\sum_{k=1}^{3} |t_k - b_k|}{2|r - l|} \tag{2}$$

The calibrated EAR and MAR thresholds are:

$$\tau_E = \text{clip}\left(\max(\mu_E - 2.2\sigma_E,\ 0.72\mu_E),\ 0.18,\ 0.24\right) \tag{3}$$

$$\tau_M = \max(0.55,\ \mu_M + 3\sigma_M) \tag{4}$$

PERCLOS is calculated over a sample window as:

$$PERCLOS = \frac{100}{N} \sum_{i=1}^{N} c_i, \qquad c_i \in \{0, 1\} \tag{5}$$

Here, c_i represents a binary value indicating whether frame i has been classified as eye-closed or not.

During calibration, the smallest EAR decile is excluded before calculating the cleaned mean and standard deviation.

D. Decision Rules

The decision rules implemented in the application are presented in Table III.

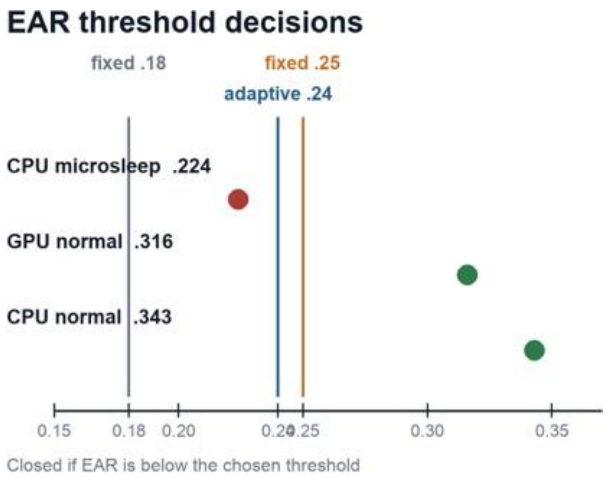


Fig. 2. Geometric measurements and adaptive thresholds used by the implementation.

TABLE III
DECISION RULES

Rule	Implementation Condition
Eye frame	$EAR < \tau_E$ OR $P(\text{closed}) > 0.75$.
Microsleep	35 consecutive closed frames.
Yawn	$MAR \geq \tau_M$ for 28 frames.
Head nod	Pitch $\leq -18^{\circ}$ for 22 frames.
High PERCLOS	At least 300 samples and PERCLOS $\geq 15\%$.

VI. DATASET, SOFTWARE AND REPRODUCIBILITY

A. Dataset Status

The repository contains `ocunet.xml`, `ocunet.bin`, and a training script. The script expects the following training arrays:

- `dataset/x_train.npy`
- `dataset/y_train.npy`
- `dataset/x_val.npy`
- `dataset/y_val.npy`

These arrays and their descriptions are missing from the repository.

The final prototype therefore uses the supplied pretrained OcuNet IR weights. The available files do not identify the original eye datasets, subject partitions, or augmentation procedures.

A confusion matrix cannot be reconstructed from model weights alone. The paper consequently does not claim to reproduce the model's original training accuracy.

The project source repository is:
https://github.com/Astron3ts/Adaptive-Driver-Drowsiness-Detection-using-OpenVINO

VII. HARDWARE-AWARE IMPLEMENTATION

A. CPU Demonstration

The hardware-specific demonstration was performed on an HP Victus 15-fa1xxx laptop running Windows 11 with 16 GB RAM and an NVIDIA GeForce RTX 4050.

The five CPU screenshots were captured on this machine with the interface explicitly displaying CPU as the inference target for the OpenVINO networks.

The HUD's "Infer" value measures only the time spent running the face detector. It excludes MediaPipe landmark extraction, OcuNet computation, rendering, and camera input/output operations.

The displayed FPS value is determined over each individual loop iteration. It therefore depends on camera frame delivery, face processing, and interface rendering.

TABLE IV
DATA ASSETS AND CANDIDATE BENCHMARK DATASETS

Dataset	Description and Project Status
OcuNet arrays	Expected 34×26 eye images and binary labels. Not bundled.
NTHU-DDD	Day/night videos, eyewear, and multiple drowsy behaviours. Reviewed candidate.
DROZY	Video and vigilance information. Reviewed candidate.
YawDD	Yawning and facial-behaviour videos. Reviewed candidate.
NITYMED	Eye and yawn material used in recent embedded work. Reviewed candidate.

TABLE V
CPU, GPU AND FPGA STATUS

Target	Status and Evidence
HP Victus CPU	Implemented and screenshot-tested. 28.7–39.1 displayed FPS.
RTX 4050 CUDA	Available for future PyTorch training. Original arrays and logs are required.
OpenVINO GPU	Supported when an eligible device is enumerated. No verified RTX 4050 benchmark is available.
FPGA	Conceptual architecture only. Quantisation and hardware validation are required.

B. GPU Path

The PyTorch training script can select CUDA if a compatible GPU is detected. This means that the RTX 4050 could support future OcuNet training.

However, the submitted OpenVINO screenshots do not show the RTX 4050 as the inference target. Therefore, it is treated as available hardware rather than a measured OpenVINO performance result.

C. Possible FPGA Deployment

An FPGA-based implementation would use a heterogeneous architecture. Camera data would be transferred through DMA into a frame buffer.

Image resizing, normalisation, and convolution-heavy face and eye processing could be quantised and mapped to the DSP units and internal memory buffers of the FPGA.

The remaining operations, including landmark post-processing, EAR/MAR calculations, `solvePnP`, counters, interface rendering, and audio, would remain on embedded processors or a host CPU.

TABLE VI
INDIVIDUAL MEMBER CONTRIBUTION

Member	Recorded Contribution
Shivam Tyagi	Classical drowsiness methods, EAR and PERCLOS study, adaptive-threshold analysis, evaluation planning, macOS runtime demonstration, screenshot interpretation, and final report preparation.
Akash A Gaddi	Deep-learning and edge-inference review, OpenVINO study, project repository, and integration of the supplied face-detector and OcuNet model assets.
Joint work	Topic selection, comparison of 20 journal papers, research-gap definition, final hybrid-model selection, and viva preparation.

TABLE VII
SCREENSHOT-DERIVED CPU READINGS

State	Infer ms	FPS	Visible Output
Calibration	11.6	32.0	CPU; 53 s remaining.
Normal	6.2	38.4	EAR 0.343 > 0.240; OcuNet 63%.
Microsleep	7.7	28.7	EAR 0.224 < 0.240; OcuNet 82%.
Yawn	5.2	36.8	MAR 1.125.
Head nod	6.1	39.1	Pitch -178.8°.
Mean	7.36	35.0	Five captured frames.

Calibration using representative data would be necessary to establish the INT8 ranges. The next stages would include post-quantisation accuracy checking, synthesis, timing closure, and measurement of the complete round-trip latency.

This architecture was not implemented in the present prototype.

VIII. PROJECT PLAN AND MEMBER CONTRIBUTION

The project timeline originally covered the period from 1 August to 19 October 2026.

The activity initially planned for 3–10 October was changed from untested INT8 benchmarking to cross-platform validation, controlled warning capture, and evidence preparation.

This adjustment allowed the project to focus on demonstrating the functionality already completed and supported by the available screenshots.

IX. EXPERIMENTAL METHODOLOGY AND PERFORMANCE METRICS

A. Demonstration Protocol

Five controlled operating states were captured:

1) Baseline calibration.
2) Normal monitoring.
3) Sustained eye closure.
4) Prolonged mouth opening.
5) Downward head pose.

Each test case included the interface, face box, and relevant warning. The objective was to verify whether the intended decision branch appeared and whether the visible cue values corresponded to the threshold values defined in the source code.

This procedure does not measure population-level sensitivity or specificity.

B. Performance Metrics

The interface displays the device name, face-detector inference time, instantaneous FPS, and peak FPS.

A complete evaluation should additionally include accuracy, precision, recall, F1-score, false alarms per hour, and detection delay.

Accuracy alone may conceal classification bias. End-to-end timing should be measured from the moment a video frame is acquired until the warning is displayed. The timing results should be reported as a distribution after the system has warmed up.

Memory usage, model size, and energy consumption per frame would also be useful when comparing hardware platforms.

These additional metrics are proposed for future evaluation. No values are reported without corresponding predictions and ground-truth labels.

C. Evaluation Conditions and Validity

All five CPU screenshots were collected using the same HP Victus laptop, its built-in camera, and the same live demonstration application.

After the initial sixty seconds of baseline capture, the operator intentionally produced the visual responses required to activate the detection branches.

Although the screenshots confirm that the corresponding application branches are operational, they do not represent a repeatable driver study. Individual FPS and inference readings must not be treated as confidence intervals or average hardware measurements.

A more reliable evaluation would use a predetermined labelled video set. The same videos could be played back to both CPU and GPU implementations, removing differences caused by live camera input.

Each generated output image could be timestamped and matched with a ground-truth label. This would allow precision, recall, F1-score, false alarms per hour, and detection delay to be calculated separately for microsleep, yawning, head-nodding, and PERCLOS warnings.

The test set should also include different drivers, eyewear, varying illumination, partial face occlusions, and different camera viewpoints.

X. Results and Discussion

A. Calibration and Normal Monitoring

The calibration screenshot shows that Phase 1 had been completed partially, with 53 seconds remaining. Face boxes and facial landmark points are visible, indicating that the application was sampling EAR and MAR values.

The CPU inference target is explicitly displayed. The face-detector time for this captured iteration was 11.6 ms, with a displayed frame rate of 32.0 FPS.

After the 60-second calibration period, the implementation removes the lowest EAR decile, calculates the cleaned mean and standard deviation, clips the eye threshold between 0.18 and 0.24, and determines the mouth threshold.

In normal monitoring, the EAR value was 0.343, which was greater than the calibrated threshold of 0.240.

OcuNet reported a 63% closed-eye probability, below the 75% threshold. PERCLOS was 0.00%, and the interface displayed "Status: Alert".

The label is intended to indicate an awake state. However, "Status: Awake" would communicate this condition more clearly.

The face-detection time was 6.2 ms and the displayed frame rate was approximately 38.4 FPS.

B. Microsleep Branch

The microsleep screenshot shows an EAR value of 0.224, which is below the calibrated threshold of 0.240.

OcuNet reported an 82% closed-eye probability, exceeding the 75% decision threshold. Both eye cues therefore indicated a closed eye.

PERCLOS was 13.1%, which was below the 15% threshold for the longer-window warning.

However, the system displayed a microsleep warning because the 35-frame consecutive-closure condition had been reached.

This illustrates the distinction between the two mechanisms. The microsleep counter detects one prolonged period of eye closure, whereas PERCLOS measures eye closure across a larger sample window.

The face-detector time was 7.7 ms and the displayed FPS was 28.7.

C. Yawn Branch

The MAR value in the yawn screenshot was 1.125, exceeding the dynamically calculated threshold of 0.55.

The warning appeared after a sequence of high-MAR frames lasting at least 28 frames.

EAR remained at 0.311 and OcuNet reported 48%, so the eye-closure condition was not met.

The completed-yawn counter is updated after the mouth closes following a qualified event.

The face-detector time was approximately 5.2 ms and the displayed frame rate was 36.8 FPS.

D. Head-Pose Branch

The head-nod screenshot demonstrates execution of the warning branch after the pitch condition persisted for 22 frames.

The displayed pitch value was -178.8°. This value reflects Euler-angle wrapping near $\pm 180^{\circ}$.

Although the warning branch is visibly activated, the angle representation needs correction before it can be used as a reliable physical measurement.

A production-quality implementation should normalise and centre the Euler-angle representation relative to a calibrated neutral orientation.

E. Runtime Interpretation

Across the five CPU screenshots, the face-detector times ranged from 5.2 to 11.6 ms.

The mean was 7.36 ms and the median was 6.2 ms.

The displayed frame rates ranged from 28.7 to 39.1 FPS, with a mean of 35.0 FPS.

These readings show that the live demonstration remained interactive on the HP Victus CPU during the captured states. They do not represent repeated-trial performance measurements.

F. Comparison with Published Work

The design follows previous recommendations to combine multiple visual cues and temporal evidence.

It uses interpretable measurements similar to earlier PERCLOS and fuzzy-fusion systems [2], [13], while also incorporating adaptive thresholds studied in EAR-based approaches [8], [17].

Compared with 3D CNN, LSTM, and graph-based approaches [6], [10], [16], [18], [21], the proposed implementation uses less learned temporal computation and instead relies on frame-based persistence thresholds.

Therefore, the primary contribution is architectural: an explainable real-time demonstration integrating several indicators on a laptop CPU.

A detection-accuracy comparison with published work remains unavailable until the proposed system is evaluated using a standardised, labelled, subject-independent dataset.

XI. Additional Comparisons

A. CPU and GPU Observations

The five CPU screenshots in Table VII show 28.7–39.1 FPS and 5.2–11.6 ms face-detector inference time.

Three additional screenshots from the same laptop show OpenVINO GPU.0 at 40.0–44.3 FPS and 4.4–4.8 ms.

These values are individual screenshot observations and not repeated-trial averages.

The on-screen inference timer records face-detector execution, whereas FPS measures the complete application loop.

The capture moments and application states were different. Therefore, the readings indicate a possible GPU execution path but do not prove a speedup.

TABLE VIII
STATE-BY-STATE EVIDENCE INTERPRETATION

State	Observed Evidence	Supported Conclusion	Remaining Limitation
Calibration	Phase 1 label, timer, and landmarks.	Baseline acquisition is active.	No claim about threshold quality from one subject.
Normal	EAR and OcuNet remain below the closure decision.	Monitoring path and interface are functional.	The displayed "Alert" label is ambiguous.
Microsleep	Both eye cues cross their thresholds.	Sustained-closure warning branch executed.	Duration is inferred from the branch, not externally timed.
Yawn	High MAR and warning text.	Mouth-duration branch executed.	Yawn count updates only after closure.
Head nod	Negative pitch and orange warning.	Pose branch executed.	Angle wrapping requires correction.

OpenVINO identifies GPU.0 as an Intel GPU device. The screenshots do not establish that inference ran on the laptop's NVIDIA GeForce RTX 4050 [31].

B. GPU Screenshot Evidence

The following screenshots provide additional evidence of the OpenVINO GPU.0 execution path.

They show the normal monitoring state, yawn warning, and head-nod warning respectively.

C. EAR Threshold Calibration

The source code calibrates the EAR threshold for each user and restricts the resulting value to the range 0.18–0.24.

The available screenshot shows an adaptive threshold of 0.240.

For the CPU microsleep observation, EAR equals 0.224. A fixed threshold of 0.18 would treat this measurement as open, whereas both the adaptive 0.24 threshold and fixed 0.25 threshold would classify it as closed.

Normal readings remain open under all three threshold settings.

The available screenshots cannot establish whether 0.24 or 0.25 performs better because they contain no EAR observations between those two values.

Previous research has also reported different preferred fixed thresholds depending on the selected dataset [19].

D. OcuNet vs. Other Eye Models

Florez *et al.* compared InceptionV3, VGG16, ResNet50V2, and DD-AI using their eye dataset and a Jetson Nano platform [20].

Figure **??** presents the published accuracy comparison and the separate comparison of reference architecture sizes.

OcuNet contains 25,666 trainable parameters, making it considerably smaller than the standard reference architectures displayed in the comparison.

The supplied OpenVINO weights are also integrated into the live EAR and warning pipeline.

However, the screenshots only demonstrate that OcuNet executes within the application. They do not establish that its accuracy exceeds the published models.

Hashemi *et al.* also examined custom CNN and VGG-based eye classifiers in related work [9].

XII. LIMITATIONS AND SAFETY CONSIDERATIONS

The main limitation of the study is the missing provenance of the OcuNet training and evaluation data.

The repository contains executable model files, but it does not identify the original eye datasets, subject partitions, or data augmentation methods used during training.

Another limitation is that the screenshots represent one subject performing controlled gestures. They do not evaluate the effects of glasses, low-light conditions, partial face occlusions, vehicle vibration, road motion, or demographic variation.

The face-detector timer also represents only part of the total processing time. It is not an end-to-end latency measurement.

The head-pose angle wrapping requires correction and validation before it can reliably support production decisions.

Finally, the false-alarm rate and the suitability of warning frequency for safe human-machine interaction have not been measured.

The present prototype must not be treated as a certified driver-safety device.

Future work should establish a fixed labelled video dataset and create subject-independent training, validation, and test partitions.

If the model is retrained using the RTX 4050, the dataset manifest, random seed, epoch log, and best checkpoint should be preserved.

Since heterogeneous computing architectures may improve inference time and energy efficiency, an FPGA implementation is also worth investigating. Such a study should report post-quantisation accuracy, resource utilisation, clock frequency, power consumption, and end-to-end latency.

XIII. CONCLUSION

This project demonstrates a functional webcam-based prototype for driver drowsiness detection. The implementation combines personal calibration, geometric eye and mouth measurements, a compact CNN, head-pose estimation, and temporal evidence.

Controlled screenshots show successful activation of calibration, normal monitoring, microsleep, yawning, and head-nod branches on the HP Victus Windows laptop.

Across five CPU captures, the displayed frame rates ranged from 28.7 to 39.1 FPS, while face-detector calls took between

5.2 and 11.6 ms. These values describe individual observations and do not represent an experimental accuracy study or repeated-trial benchmark.

The prototype demonstrates the feasibility of combining adaptive thresholds, learned eye-state information, and temporal decision rules in a laptop-based application.

The report also documents the missing provenance of the supplied OcuNet weights and the absence of labelled evaluation data. The RTX 4050 is considered available hardware rather than a measured OpenVINO inference target, while FPGA deployment remains a proposed future direction.

ACKNOWLEDGMENT

The authors acknowledge Dr. J. Saira Banu, School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, for collaboration and guidance during the project.

REFERENCES

[1] Q. Ji, Z. Zhu, and P. Lan, "Real-time nonintrusive monitoring and prediction of driver fatigue," *IEEE Transactions on Vehicular Technology*, vol. 53, no. 4, pp. 1052–1068, 2004, doi: 10.1109/TVT.2004.830974.

[2] L. M. Bergasa, J. Nuevo, M. A. Sotelo, R. Barea, and M. E. Lopez, "Real-time system for monitoring driver vigilance," *IEEE Transactions on Intelligent Transportation Systems*, vol. 7, no. 1, pp. 63–77, 2006, doi: 10.1109/TITS.2006.869598.

[3] L. Zhao, Z. Wang, X. Wang, and Q. Liu, "Driver drowsiness detection using facial dynamic fusion information and a DBN," *IET Intelligent Transport Systems*, vol. 12, no. 2, pp. 127–133, 2018, doi: 10.1049/iet-its.2017.0183.

[4] G. Sikander and S. Anwar, "Driver fatigue detection systems: A review," *IEEE Transactions on Intelligent Transportation Systems*, vol. 20, no. 6, pp. 2339–2352, 2019, doi: 10.1109/TITS.2018.2868499.

[5] M. Ramzan, H. U. Khan, S. M. Awan, A. Ismail, M. Ilyas, and A. Mahmood, "A survey on state-of-the-art drowsiness detection techniques," *IEEE Access*, vol. 7, pp. 61904–61919, 2019, doi: 10.1109/ACCESS.2019.2914373.

[6] J. Yu, S. Park, S. Lee, and M. Jeon, "Driver drowsiness detection using condition-adaptive representation learning framework," *IEEE Transactions on Intelligent Transportation Systems*, vol. 20, no. 11, pp. 4206–4218, 2019, doi: 10.1109/TITS.2018.2883823.

[7] W. Deng and R. Wu, "Real-time driver-drowsiness detection system using facial features," *IEEE Access*, vol. 7, pp. 118727–118738, 2019, doi: 10.1109/ACCESS.2019.2936663.

[8] C. B. S. Maior, M. J. C. Moura, J. M. M. Santana, and I. D. Lins, "Real-time classification for autonomous drowsiness detection using eye aspect ratio," *Expert Systems with Applications*, vol. 158, art. 113505, 2020, doi: 10.1016/j.eswa.2020.113505.

[9] M. Hashemi, A. Mirrashid, and A. Beheshti Shirazi, "Driver safety development: Real-time driver drowsiness detection system based on convolutional neural network," *SN Computer Science*, vol. 1, art. 289, 2020, doi: 10.1007/s42979-020-00306-9.

[10] J. S. Wijnands, J. Thompson, K. A. Nice, G. D. P. A. Aschwanden, and M. Stevenson, "Real-time monitoring of driver drowsiness on mobile platforms using 3D neural networks," *Neural Computing and Applications*, vol. 32, pp. 9731–9743, 2020, doi: 10.1007/s00521-019-04506-0.

[11] Y. Ed-Doughmi, N. Idrissi, and Y. Hbali, "Real-time system for driver fatigue detection based on a recurrent neuronal network," *Journal of Imaging*, vol. 6, no. 3, art. 8, 2020, doi: 10.3390/jimaging6030008.

[12] Z. Zhao, N. Zhou, L. Zhang, H. Yan, Y. Xu, and Z. Zhang, "Driver fatigue detection based on convolutional neural networks using EM-CNN," *Computational Intelligence and Neuroscience*, vol. 2020, art. 7251280, 2020, doi: 10.1155/2020/7251280.

[13] A. Celecia, K. Figueiredo, M. Vellasco, and R. Gonzalez, "A portable fuzzy driver drowsiness estimation system," *Sensors*, vol. 20, no. 15, art. 4093, 2020, doi: 10.3390/s20154093.

[14] A. Moujahid, F. Dornaika, I. Arganda-Carreras, and J. Reta, "Efficient and compact face descriptor for driver drowsiness detection," *Expert Systems with Applications*, vol. 168, art. 114334, 2021, doi: 10.1016/j.eswa.2020.114334.

[15] M. Dua, Shakshi, R. Singla, S. Raj, and A. Jangra, "Deep CNN models-based ensemble approach to driver drowsiness detection," *Neural Computing and Applications*, vol. 33, no. 8, pp. 3155–3168, 2021, doi: 10.1007/s00521-020-05209-7.

[16] A. Quddus, A. S. Zandi, L. Prest, and F. J. E. Comeau, "Using long short term memory and convolutional neural networks for driver drowsiness detection," *Accident Analysis and Prevention*, vol. 156, art. 106107, 2021, doi: 10.1016/j.aap.2021.106107.

[17] A.-C. Phan, N.-H.-Q. Nguyen, T.-N. Trieu, and T.-C. Phan, "An efficient approach for detecting driver drowsiness based on deep learning," *Applied Sciences*, vol. 11, no. 18, art. 8441, 2021, doi: 10.3390/app11188441.

[18] J. Bai, W. Yu, Z. Xiao, V. Havyarimana, A. C. Regan, H. Jiang, and L. Jiao, "Two-stream spatial-temporal graph convolutional networks for driver drowsiness detection," *IEEE Transactions on Cybernetics*, vol. 52, no. 12, pp. 13821–13833, 2022, doi: 10.1109/TCYB.2021.3110813.

[19] C. Dewi, R.-C. Chen, C.-W. Chang, S.-H. Wu, X. Jiang, and H. Yu, "Eye aspect ratio for real-time drowsiness detection to improve driver safety," *Electronics*, vol. 11, no. 19, art. 3183, 2022, doi: 10.3390/electronics11193183.

[20] R. Florez, F. Palomino-Quispe, A. B. Alvarez, R. J. Coaquira-Castillo, and J. C. Herrera-Levano, "A real-time embedded system for driver drowsiness detection based on visual analysis of the eyes and mouth using convolutional neural network and mouth aspect ratio," *Sensors*, vol. 24, no. 19, art. 6261, 2024, doi: 10.3390/s24196261.

[21] L. Lin, S. Wang, J. Yang, and F. Wei, "A multi-aware graph convolutional network for driver drowsiness detection," *Knowledge-Based Systems*, vol. 305, art. 112643, 2024, doi: 10.1016/j.knosys.2024.112643.

[22] D. Salem and M. Waleed, "Drowsiness detection in real-time via convolutional neural networks and transfer learning," *Journal of Engineering and Applied Science*, vol. 71, art. 122, 2024, doi: 10.1186/s44147-024-00457-z.

[23] L. He, L. Zhang, Q. Sun, and X. Lin, "A generative adaptive convolutional neural network with attention mechanism for driver fatigue detection with class-imbalanced and insufficient data," *Behavioural Brain Research*, vol. 464, art. 114898, 2024, doi: 10.1016/j.bbr.2024.114898.

[24] Y. Sun, R. Wang, H. Zhang, N. Ding, S. Ferreira, and X. Shi, "Driving fingerprinting enhances drowsy driving detection: Tailoring to individual driver characteristics," *Accident Analysis and Prevention*, vol. 208, art. 107812, 2024, doi: 10.1016/j.aap.2024.107812.

[25] G. Liu, K. Wu, W. Lan, and Y. Wu, "YOLO-FDCL: Improved YOLOv8 for driver fatigue detection in complex lighting conditions," *Sensors*, vol. 25, no. 15, art. 4832, 2025, doi: 10.3390/s25154832.

[26] L. El Sahmarany, M. Alkhaldi, and S. I. Alzahrani, "Sensing technologies and physiological parameters for real-time driver drowsiness detection: A comprehensive review," *Sensors*, vol. 26, no. 11, art. 3333, 2026, doi: 10.3390/s26113333.

[27] T. Soukupova and J. Cech, "Real-time eye blink detection using facial landmarks," in *Proc. 21st Computer Vision Winter Workshop*, Rimske Toplice, Slovenia, 2016, pp. 1–8.

[28] Y. Kartynnik, A. Ablavatski, I. Grishchenko, and M. Grundmann, "Real-time facial surface geometry from monocular video on mobile GPUs," arXiv:1907.06724, 2019.

[29] OpenVINO Contributors, "Supported devices and inference modes," OpenVINO 2025 Documentation, accessed Oct. 14, 2026. [Online]. Available: https://docs.openvino.ai/2025/documentation/compatibility-and-support/supported-devices.html.

[30] S. Abtahi, M. Omidyeganeh, S. Shirmohammadi, and B. Hariri, "YawDD: A yawning detection dataset," in *Proc. 5th ACM Multimedia Systems Conf.*, 2014, pp. 24–28, doi: 10.1145/2557642.2563678.

[31] OpenVINO Contributors, "GPU Device," OpenVINO 2025 Documentation. [Online]. Available: https://docs.openvino.ai/2025/openvino-workflow/running-inference/inference-devices-and-modes/gpu-device.html.

[32] Keras, "Keras Applications," parameter counts for standard InceptionV3, ResNet50V2, and VGG16 architectures. [Online]. Available: https://keras.io/api/applications/.

X.RESULTS AND DISCUSSION

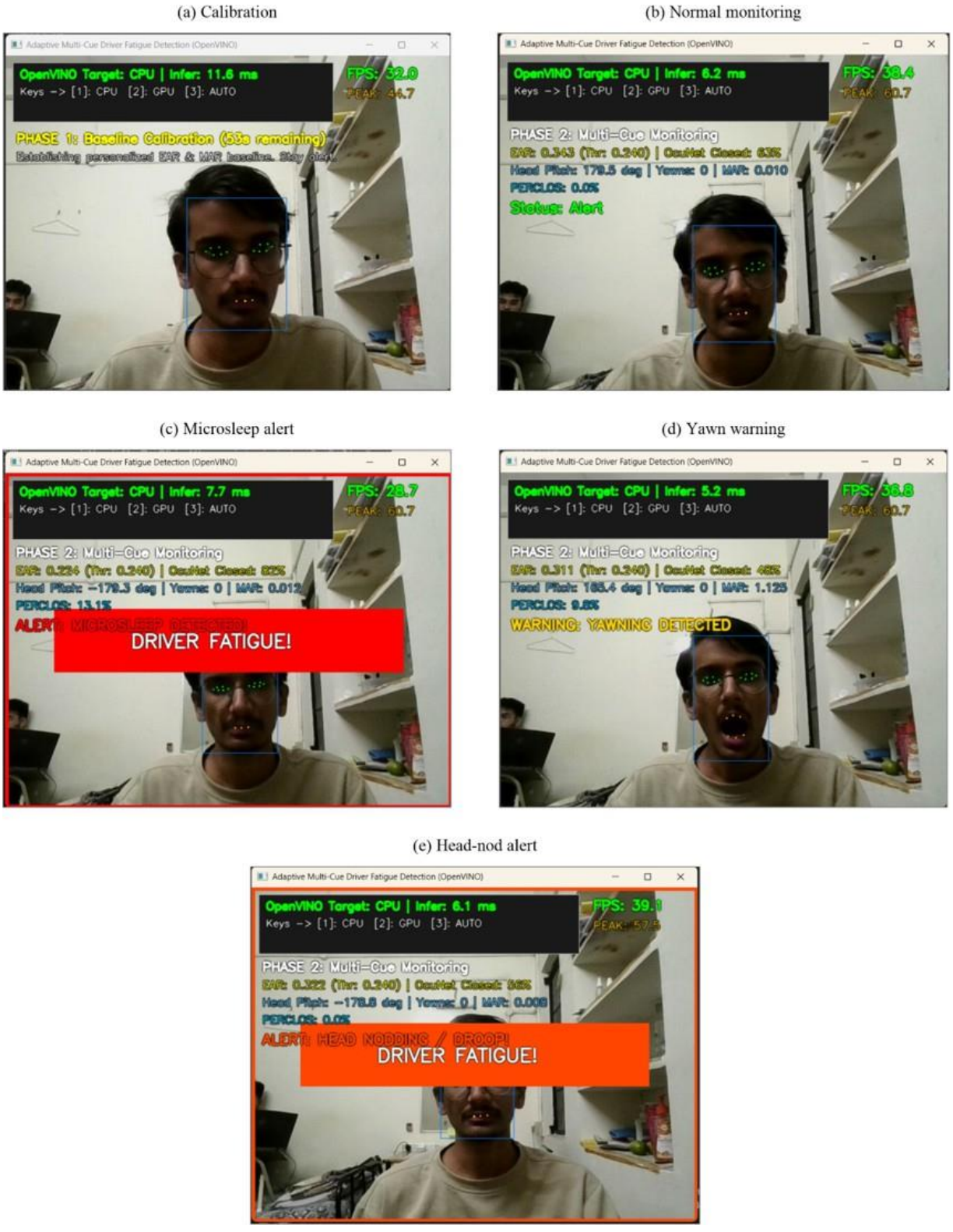


Fig. 4. Controlled live states: (a) calibration, (b) normal monitoring, (c) microsleep alert, (d) yawn warning, and (e) head-nod alert.

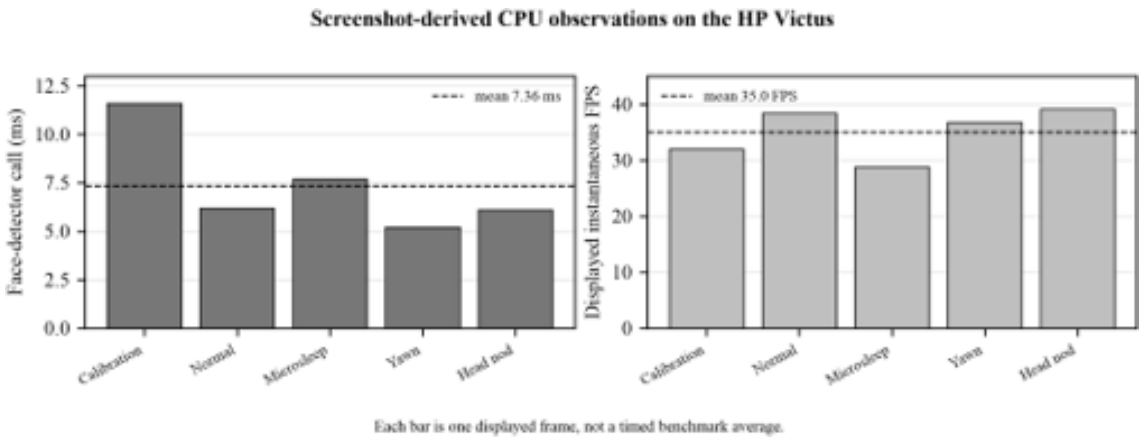


Fig. 5. Screenshot-derived CPU observations on the HP Victus laptop. The bars represent individual displayed frames rather than benchmark averages.

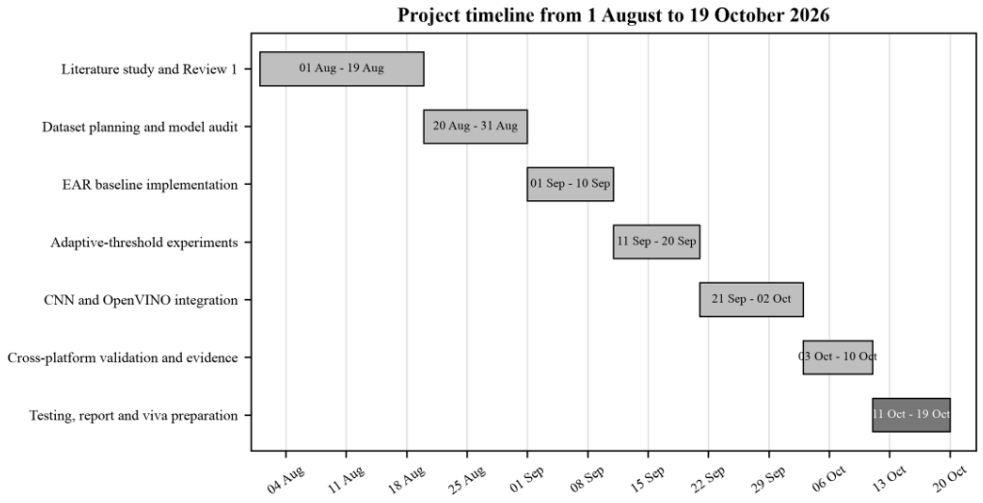


Fig. 6. Project timeline from 1 August to 19 October 2026.

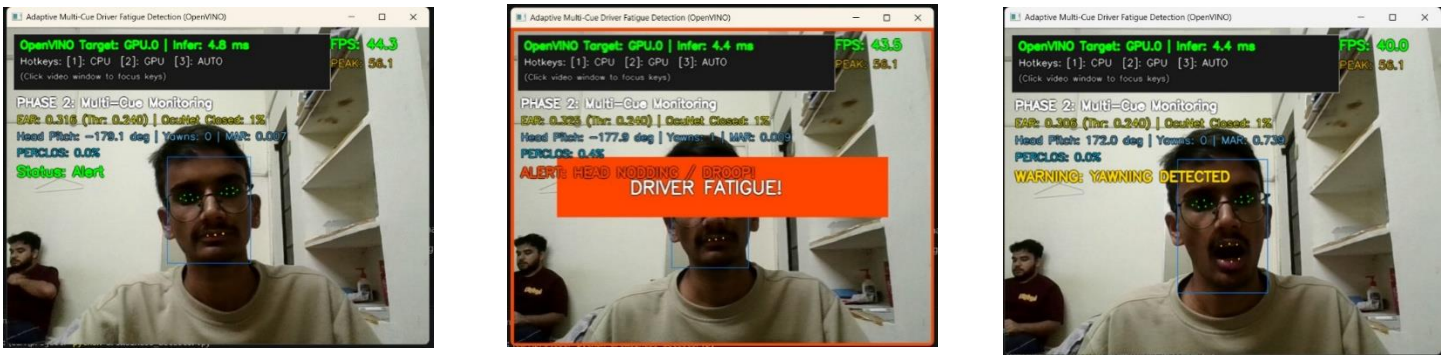


Fig. 8. Additional live GPU.0 screenshots showing the monitoring and warning states. (a) Normal monitoring, 44.3 FPS(b) Yawn warning, 40.0 FPS(c) Head-nod warning, 43.5 FPS